

Assignment 2

AI-Assisted Network Security Analysis Bug Bounty Exploration

Tareq Alsubaie
550258025

Bharathwaj Murali
550086211

Peisen Zheng
540660793

Nishanth Shanmugasundaram
550258933

Shyam Karthick Venkatraman Balakrishnan
550224271

1 Introduction

The provided APK is an Android application named `a2_case1` and in it a package `com.example.mastg_test0019` that displays remote web content inside an embedded `WebView`. The Part A analysis is focused strictly on the intended vulnerability class in the assignment specification: insecure network transport and broken TLS validation that could enable a Man-in-the-Middle (MITM) attack. Static analysis shows that the application permits cleartext traffic, loads a remote page over HTTP, and overrides SSL error handling so that invalid certificates are still accepted. Together, these behaviours create a direct transport security weakness that allows an on-path attacker to intercept and manipulate network content.[1]

2 Decompilation Summary

The APK was successfully decompiled using JADX. The decompilation recovered the Android manifest, the launcher activity, the layout files, and the application source required for network-focused analysis. The manifest confirms the package name and app-level transport settings, while the recovered `MainActivity` contains the network-relevant behavior. The layout also confirms the presence of a `WebView`, which is the component used to load and render remote content.

The recovered code is sufficient for Part A because it exposes the entry point, the transport configuration, and the concrete implementation of the insecure network behavior. In particular, the analysis was based on the manifest configuration and the logic inside `MainActivity`, where the `WebView`, `WebViewClient`, and SSL error handling are implemented.

3 System and Threat Model

The application can be modeled as a client that requests remote web content and renders it inside a `WebView`. The communication path is simple: the user interacts with the Android app, the app sends a request across the Internet, and a remote web server returns content that is displayed within the app. This makes the network a critical trust boundary, because data

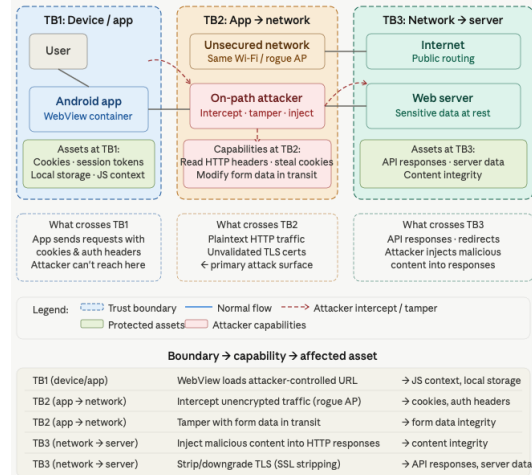


Figure 1: Threat Model with explicit trust boundaries

crossing that boundary must be protected for confidentiality, integrity, and authenticity.

The main protected assets in this model are session cookies, bearer tokens, authentication headers, form data entered into the rendered page, and the integrity of the content shown to the user. If the app relies on remote content for instructions, workflows, or user interaction, then server response integrity is itself a security-critical asset.

The most relevant attacker is an on-path adversary, such as a malicious public Wi-Fi user, a rogue hotspot operator, or an attacker controlling a compromised gateway. This attacker can observe, intercept, and modify traffic between the app and the server. If traffic is sent over HTTP, the attacker can directly read and alter it because it is not encrypted. If certificate validation is bypassed, the attacker can also impersonate the server using an invalid or forged certificate and still have the connection accepted. This creates a direct and realistic path from attacker capability to asset compromise and user harm.

4 Vulnerability Analysis

The intended Part A vulnerability is an insecure transport configuration that enables a realistic man-in-the-middle (MITM) attack. The first root cause is that the application permits clear-

text network traffic. This is evident in the manifest [?]:

```
<application
...
    android:usesCleartextTraffic="true"
... >
```

This weakens Android’s default transport protections by allowing HTTP communication. The risk is realized in MainActivity, where the embedded WebView loads a cleartext URL:

```
webView.setWebViewClient(new WebViewClient() {
    @Override
    public void onReceivedSslError(WebView webView2,
        SslErrorHandler sslErrorHandler, SslError sslError) {
        sslErrorHandler.proceed(); }
});
```

Because the page is loaded over `http://`, an on-path attacker on the same Wi-Fi network or rogue access point can intercept the request, read transmitted data, and modify the returned content before it reaches the application.

The second root cause is broken TLS validation. The custom `WebViewClient` overrides SSL error handling and explicitly continues when certificate validation fails:

```
webView.setWebViewClient(new WebViewClient() { @Override
    public void onReceivedSslError(WebView webView2,
        SslErrorHandler sslErrorHandler, SslError sslError) {
        sslErrorHandler.proceed(); }
});
```

This is insecure because `proceed()` causes the `WebView` to trust invalid certificates instead of rejecting them. In practice, the app may accept self-signed, forged, expired, or hostname-mismatched certificates, which breaks server authentication and allows active interception even when HTTPS is attempted.

Together, these behaviors create a clear MITM condition. A realistic attacker on public Wi-Fi can intercept the `WebView` traffic, inject modified HTML or JavaScript, and cause attacker-controlled content to be rendered inside the trusted application interface. The impact is significant: session cookies, authentication headers, query parameters, and form data may be disclosed, while server responses can be altered to display phishing content, malicious redirects, or misleading workflows. This compromises transport confidentiality, response integrity, and server authenticity.

The decompiled source also contains a permissive `HostnameVerifier` that returns `true`, but it is not attached to an active connection in the recovered code. For that reason, the primary evidence of the live vulnerability is the active combination of cleartext HTTP usage and SSL error bypass in the `WebView`.

5 Mitigation

The mitigations should be mapped directly to the two root causes. The first root cause is that the app permits unauthenticated plaintext transport. To fix this, the application should enforce HTTPS for all remote communication and should not load content over `http://`. In addition, cleartext traffic

should be disabled at the platform level so that insecure transport is blocked by default. These changes remove the attacker’s ability to passively read or actively modify traffic in transit on an untrusted network.

The second root cause is that the app overrides certificate validation and trusts invalid certificates. To fix this, the insecure SSL error override should be removed, and the application should use the default secure behavior or explicitly cancel requests when certificate validation fails. This ensures that self-signed, forged, expired, or mismatched certificates are rejected rather than silently trusted. That restores the server authentication property that TLS is supposed to provide.

A stronger implementation would also define a Network Security Config so that transport security policy is centralized and cleartext communication is disallowed by configuration. If the backend is stable and controlled, certificate pinning could be added as an extra defense. Pinning is not required to fix the basic weakness, but it would further reduce MITM risk by limiting which certificates the app will accept.

Overall, these mitigations are effective because they address the root causes directly: they stop the app from using unauthenticated plaintext transport and prevent it from trusting invalid certificates. As a result, they restore confidentiality, integrity, and authenticity for network communication.

6 Conclusion

The provided APK contains a network security vulnerability that falls squarely within the intended Part A scope. The app explicitly allows cleartext traffic, loads remote content over HTTP, and bypasses SSL certificate validation in its `WebView`. These behaviors allow a realistic on-path attacker to intercept, read, and modify traffic, enabling MITM attacks, content injection, response tampering, session theft, and data exposure.

The decompiled code provides strong and direct evidence for this conclusion, and the vulnerability maps precisely to the insecure transport and broken TLS validation class identified in the assignment specification. Enforcing HTTPS, disabling cleartext traffic, and preserving proper certificate validation would remove the core transport security weakness and significantly improve the app’s resistance to MITM attacks.

References

- [1] T. Alsubaie, “INFO5995-A2,” GitHub repository, 2026. [Online]. Available: <https://github.com/TareqAlsubaie/INFO5995-A2>